

Curso <i>Engenharia de Software</i> <i>Programação Orientada a Objetos</i>	Campus <i>Rio Verde</i>
Professor <i>Sergio Souza Novak</i>	Valor 7,0
Data da Avaliação 20/03/2026	Código ESW448

QUESTÕES COM RESOLUÇÕES

1. **(ENADE 2017):** O encapsulamento é um mecanismo da programação orientada a objetos no qual os membros de uma classe (atributos e métodos) constituem uma caixa preta. O nível de visibilidade dos membros pode ser definido pelos modificadores de visibilidade “privado”, “público” e “protegido”. Com relação ao comportamento gerado pelos modificadores de visibilidade, assinale a opção correta.
- A) Um atributo privado pode ser acessado pelos métodos privados da própria classe e pelos métodos protegidos das suas classes descendentes.
 - B) Um atributo privado pode ser acessado pelos métodos públicos da própria classe e pelos métodos públicos das suas classes descendentes.
 - C) Um membro público é visível na classe à qual ele pertence, mas não é visível nas suas classes descendentes.
 - D) Um método protegido não pode acessar os atributos privados e declarados na própria classe.
 - E) **Um membro protegido é visível na classe à qual pertence e em suas classes descendentes. ✓**

Resolução

Gabarito: E

Análise das alternativas:

- **A — Falsa:** Um atributo `private` **não** pode ser acessado por subclasses, nem mesmo por seus métodos `protected`. O acesso `private` é restrito à própria classe.
- **B — Falsa:** Atributo `private` **não** é acessível por descendentes, independentemente do modificador do método da subclasse.
- **C — Falsa:** Um membro `public` é visível em qualquer lugar, inclusive nas classes descendentes.
- **D — Falsa:** Um método `protected` **pode** acessar atributos `private` da *própria* classe normalmente.
- **E — Verdadeira:** O modificador `protected` garante visibilidade na própria classe e em todas as suas subclasses (classes descendentes), mesmo em pacotes diferentes.

2. **Encapsulamento:** Implementação da classe **ContaBancaria** com o atributo **saldo** privado e os métodos depositar, sacar, getSaldo e construtor.

Resolução

```
public class ContaBancaria {
    private double saldo;

    // Construtor
    public ContaBancaria(double saldoInicial) {
        if (saldoInicial >= 0) {
            this.saldo = saldoInicial;
        } else {
            this.saldo = 0;
        }
    }

    // Deposita um valor na conta
    public void depositar(double valor) {
        if (valor > 0) {
            this.saldo += valor;
        }
    }

    // Saca um valor da conta, se houver saldo
    public void sacar(double valor) {
        if (valor > 0 && valor <= this.saldo) {
            this.saldo -= valor;
        }
    }

    // Retorna o saldo atual
    public double getSaldo() {
        return this.saldo;
    }
}
```

Justificativa:

- O atributo saldo é declarado private, garantindo encapsulamento — nenhuma classe externa pode modificá-lo diretamente.
- O construtor recebe o saldo inicial e valida que não seja negativo.
- depositar e sacar validam os valores antes de alterar o saldo, protegendo a integridade do objeto.
- getSaldo() é o único ponto de leitura do saldo (getter), mantendo o controle sobre o acesso ao dado.

3. Na programação orientada a objetos, os modificadores de acesso controlam a visibilidade de classes, atributos e métodos. Assinale a alternativa que apresenta o modificador que permite que um membro seja acessado apenas dentro da própria classe e por suas subclasses:

A) public

- B) private
- C) protected ✓
- C) package-private
- D) static

Resolução

Gabarito: C — protected

- public: acessível por **qualquer** classe.
- private: acessível **apenas** dentro da própria classe (subclasses **não** têm acesso).
- protected: acessível dentro da própria classe **e** por suas subclasses (também acessível dentro do mesmo pacote em Java). É o gabarito correto.
- package-private (sem modificador em Java): acessível apenas dentro do mesmo pacote, mas **não** por subclasses em outros pacotes.
- static: não é um modificador de acesso; define que o membro pertence à classe, não ao objeto.

4. **Construtores:** Qual é a principal finalidade de um método construtor em uma classe?

- A) Destruir o objeto quando ele não for mais necessário.
- B) Inicializar os atributos de um objeto no momento de sua criação. ✓**
- B) Permitir que o objeto seja herdado por outras classes.
- C) Converter um objeto em uma string para exibição.
- D) Impedir que o objeto seja clonado.

Resolução

Gabarito: B

O construtor é um método especial chamado automaticamente no momento em que um objeto é criado com o operador new. Sua principal finalidade é **inicializar os atributos do objeto**, colocando-o em um estado válido e consistente desde o início de sua vida útil. Exemplo: `ContaBancaria c = new ContaBancaria(500.0);` — o construtor garante que o saldo começa em 500,0.

As demais alternativas descrevem:

- **A:** função do *Garbage Collector* (destruidor), não do construtor.
- **C:** função da herança (extends), não do construtor.
- **D:** função do método `toString()`.
- **E:** sem relação com o construtor.

5. **Indústria 4.0 — Máquina CNC:** Analise o código da classe `MaquinaCNC` e julgue as afirmações:

- () Se a linha 14 for removida, a máquina poderá produzir peças mesmo estando desligada.
() Os atributos das linhas 03 a 08 estão corretamente encapsulados como private.
() O método precisaManutencao expõe diretamente o atributo horasOperacao, violando o encapsulamento.

- A) V - V - V - F.
B) V - F - V - V.
C) F - V - F - V.
D) V - V - F - F. ✓
D) F - F - V - V.

Resolução

Gabarito: D — V - V - F

Análise de cada afirmação:

1ª afirmação — VERDADEIRA: A linha 14 é `if (!this.ligada){ return false; }`, que impede produção quando a máquina está desligada. Se for removida, o método produzir executará mesmo com `ligada = false`.

2ª afirmação — VERDADEIRA: Todos os atributos das linhas 03–08 (`modelo`, `horasOperacao`, `temperaturaAtual`, `temperaturaMaxima`, `ligada`, `pecasProduzidas`) são declarados `private`, o que os encapsula corretamente.

3ª afirmação — FALSA: O método `precisaManutencao()` é `public` e chama internamente o método `private verificarManutencao()`, que por sua vez acessa `horasOperacao`. O atributo **não é exposto diretamente**; o encapsulamento é preservado — apenas um resultado booleano é devolvido.

Observação: A questão lista três afirmações, portanto a sequência tem três termos: **V - V - F**.

6. Siderurgia — Alto Forno: Julgue as afirmações sobre a classe AltoForno:

- () O método `verificarFaixaTermica` é `private`, podendo ser invocado apenas por outras classes.
() Na linha 15, `controleSeguranca` é chamado para garantir que a temperatura não ultrapasse o limite.
() A produção de metal depende exclusivamente do forno estar ativo.
() O atributo `temperaturaInterna` poderia ser `public` sem afetar a segurança.

- A) V - V - F - F.
B) V - F - V - V.
C) F - V - F - F. ✓
C) V - V - V - F.
D) F - F - F - V.

Resolução

Gabarito: C — F - V - F - F

Análise de cada afirmação:

1ª afirmação — FALSA: `verificarFaixaTermica` é `private`, o que significa que só

pode ser invocado por métodos da **própria classe** AltoForno. A afirmação diz “por outras classes”, o que é o oposto — portanto FALSA.

2ª afirmação — VERDADEIRA: Dentro do método injetarCombustivel (linha 19), após incrementar a temperatura, controleSeguranca() é chamado (linha 21). Esse método desliga o forno se a temperatura ultrapassar o máximo, demonstrando controle de estado interno.

3ª afirmação — FALSA: O método produzirMetal (linha 25–32) verifica **duas** condições: ativo **E** verificarFaixaTermica(). A temperatura deve estar dentro da faixa operacional — portanto não depende *exclusivamente* de ativo.

4ª afirmação — FALSA: Tornar temperaturaInterna public permitiria que qualquer classe externa alterasse a temperatura arbitrariamente, driblando o controleSeguranca. Isso comprometeria a integridade do objeto e do equipamento.

7. (VUNESP 2021) Em programação orientada a objetos, métodos de acesso do tipo *setter* têm a finalidade primária de:

A) obter o valor de um atributo.

B) modificar o valor de um atributo. ✓

B) inicializar os valores de atributos de um objeto assim que ele é instanciado.

C) contar quantas vezes o valor de um atributo foi obtido.

D) salvar o valor de um atributo de um objeto em um banco de dados.

Resolução

Gabarito: B

- **Getter** (get . . .): obtém/retorna o valor de um atributo privado.
- **Setter** (set . . .): **modifica** o valor de um atributo privado, podendo incluir validações antes da alteração.
- **A** descreve o *getter*.
- **C** descreve o *construtor*.
- **D e E** não correspondem a nenhum padrão estabelecido de POO.

8. (FUNDATEC 2025 — GHC-RS) Considerando o código abaixo, quem pode acessar o atributo `v` diretamente:

```
package conceicao;  
public class Conceicao {  
    protected String v;  
}
```

A) Apenas as classes que estejam no mesmo pacote.

B) Apenas os métodos da própria classe.

C) As classes filhas de qualquer pacote. ✓

C) As classes filhas apenas do mesmo pacote.

D) Por qualquer classe de qualquer pacote.

Resolução

Gabarito: C

Em Java, o modificador `protected` concede acesso:

- (a) A todos os membros do **mesmo pacote** (*package access*).
- (b) A todas as **subclasses** (`extends`), **independentemente do pacote**.

Portanto, uma classe filha em **qualquer pacote** pode acessar v diretamente por herança.

Atenção: A alternativa A está incompleta (omite as subclasses de outros pacotes). A alternativa E descreve `public`. A alternativa B descreve `private`.

9. Considere a modelagem da classe `Carro` com atributo privado `velocidadeAtual`. Explique a importância de *getters* e *setters* e dê um exemplo de regra de negócio para `setVelocidade()`.

Resolução

Importância dos métodos *getter* e *setter*:

Quando um atributo é declarado `private`, nenhuma classe externa pode lê-lo ou alterá-lo diretamente. Os métodos *getter* e *setter* são a **interface controlada** para acessar e modificar esses valores. Isso traz as seguintes vantagens:

- (a) **Controle de acesso:** somente o que for exposto via métodos públicos é visível ao mundo externo. Atributos sensíveis ficam protegidos.
- (b) **Validação de dados:** o *setter* pode verificar se o valor recebido é válido antes de aplicá-lo, evitando estados inconsistentes no objeto.
- (c) **Manutenibilidade:** a lógica interna da classe pode mudar sem afetar o código que a utiliza, desde que a assinatura dos métodos permaneça a mesma.
- (d) **Rastreabilidade:** é possível adicionar log, notificações ou outros comportamentos quando um atributo é lido ou modificado, sem alterar código externo.

Exemplo de regra de negócio no `setVelocidade()`:

O método `setVelocidade()` poderia implementar a seguinte regra: “A *velocidade do carro não pode ser negativa e não pode ultrapassar 200 km/h (velocidade máxima permitida pelo veículo)*. Se o valor fornecido for inválido, a velocidade permanece inalterada e uma mensagem de aviso é registrada.”

Dessa forma, qualquer parte do sistema que tente definir uma velocidade de -10 ou de 300 km/h será bloqueada pela própria classe, garantindo que o objeto `Carro` nunca fique em um estado inconsistente com a realidade.